GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY

FACULTY OF COMPUTING

Intake 41 - IT3182 (IT/IS)

Intake 42 - CS22032 (DBA) / CS22023 (CS/SE)



AI-Based Intelligent Solution for a Real-World Problem

Project Title:MediSub: An AI-Based Student Support System for Medical Submission Guidance

GROUP NUMBER 51

FINAL REPORT

Table of Contents

FINAL REPORT 1
Table of Contents 2
List of Figures 3
Abstract 5
1. Introduction 6
2. Problem Background 6
3. Problem Statement 7
4. Objectives 7
4.1 General Objective 7
4.2 Specific Objectives 7
5. Selected AI Techniques 8
5.1 Natural Language Processing 8
5.2 TF-IDF Vectorization 8
5.3 Logistic Regression Classification 8
5.4 Confidence-Based Referral 8
5.5 Optical Character Recognition 8
5.6 Rule-Based Reasoning 8
6. Justification of AI Techniques 9
7. System Design 10
7.1 Overall Architecture 10
7.2 AI Classification Flow 10
7.3 Document Preparation and Checking Flow 10
7.4 Inputs and Outputs 11
8. Implementation Details 11
8.1 Technology Stack 11
8.2 Dataset Preparation 11
8.3 Training and Saving the Model 12
8.4 Prediction Service 12
8.5 Streamlit Interface 12
8.6 Submission Checklist 13
8.7 Document Preparation and OCR Checker 14

8.8 Privacy and Safety Messages 15
9. Testing and Results 16
9.1 Dataset Validation 16
9.2 Model Evaluation 16
9.3 Manual Question Tests 19
9.4 Document Checker Tests 19
9.5 Functional Test Summary 20
10. Limitations and Discussion 20
11. Ethical and Social Considerations 21
11.1 Privacy and Data Minimization 21
11.2 Accuracy and Reliability 21
11.3 Bias 21
11.4 Human Responsibility 21
11.5 Misuse and Security 21
12. Conclusion 22
14. Individual Contribution Sheets 24

List of Figures

Figure 1: Overall MediSub architecture 10
Figure 2:Question-classification workflow 10
Figure 3:MediSub home page 13
Figure 4:Question category and confidence result 13
Figure 5:Submission checklist page 14
Figure 6:Document upload and OCR checking result 15
Figure 7:Data Set Validation 16
Figure 8:Classification report for the five categories1 16
Figure 9:Classification report for the five categories2 17
Figure 10:Confusion matrix for the 39-question test set 18

List of Tables

Table 1: Justification of AI Techniques 9

Table 2:Inputs and Outputs 11
Table 3:Inputs and Outputs 11
Table 4: Dataset Preparation 12
Table 5:Manual Question Tests 19
Table 6:Document Checker Tests 19
Table 7:Functional Test Summary 20

Abstract

The medical-submission process can be challenging for university students, especially during illness or leave. Common difficulties include obtaining a reference number, identifying the required documents, knowing where to submit them and ensuring that forms are completed correctly. To address these issues, MediSub was developed as an AI-powered administrative support system. The completed prototype combines a Streamlit web interface, a supervised text-classification model and a rule-based document-checking component.

The group created 201 fictional questions across five categories. After removing 48 duplicate entries, 153 unique questions remained. The cleaned dataset was divided into 122 training questions and 31 testing questions. Using scikit-learn, the classification pipeline combined TF-IDF vectorization with multi-class Logistic Regression. The classifier achieved an accuracy of 87.1% and a weighted-average F1-score of 0.86. Prediction probabilities are displayed as confidence scores and results below the 0.60 threshold are flagged for verification by authorized staff.

The final interface contains five pages: Home, Ask a Question, Documents, Checklist and About and Help. Students can receive category-specific guidance, view submission instructions, download templates, upload document images and receive completeness warnings generated through OCR and rule-based checking. The document checker combines visible text extracted using Tesseract OCR with manual confirmations. However, it does not authenticate documents or approve medical submissions. The model performed well on reference-number and submission-location questions but showed weaker recall for out-of-scope questions. Overall, MediSub demonstrates how NLP, text classification, confidence-based referral, OCR and rule-based reasoning can support a real university administrative process while keeping final decision-making responsibility with authorized staff.

1. Introduction

Medical submissions are a means for students to document their absence due to illness and provide documentation for review by the University. The process is administrative, but can entail a series of related activities. A student may require to call the University Medical Centre, get a reference number, fill in a request form, write a cover letter, add a medical certificate, sign it and return the signed form to the appropriate office. If these steps are not known, a student may reach out to multiple staff members or return incomplete forms. MediSub is a targeted prototype designed to support students. It is designed to provide early guidance prior to the student contacting the responsible University Officer.

The system accepts a natural language query, guesses the category of the query being made and displays a relevant explanation and what should be done next. It also contains guidance on document preparation, a 7-item checklist for documents to be submitted and an OCR-based completeness checker to help check whether page images are uploaded.

This project was created for the group assignment of the course Essentials of Artificial Intelligence. It provides on its own more than one concept related to AI in a practical scope: Natural Language Processing, TF-IDF feature extraction, supervised classification, confidence-based decision support, Optical Character Recognition and rule-based reasoning. The program is still a research prototype. It is able to prepare, but it is not a number issuer, nor a medical evidence validator nor a decision on approval.

2. Problem Background

Students frequently have similar questions at various points in the medical-submission process. Others are interested in the preliminary steps to start. Others need help with the reference number, document list or submission location. The traditional support is quite reliant on face-to-face interactions with administrative personnel. This is still required for decisions but can lead to unnecessary multiple enquiries when the same simple instructions need to be repeated numerous times.

Another concern is if the preparation is incomplete. A student might miss a page of the request form, leave out a period of absence, miss a signature, or give a blurry picture. These issues may cause delays in review because the student will have to make edits to the documents and return them. A simple digital checking will not ensure authenticity of signature or certificate, but will serve as a reminder to the student of the visible requirements before handing over.

The initial plan was to have a small text classifier and checklist. The solution expanded to a five-page Streamlit app during development. It now integrates the trained model to guidance messages, includes a confidence threshold, supplies downloadable document templates, presents manual requirement checks and also offers an OCR ready document checking backend. No real student identities, diagnoses or medical files were used in the training dataset.

3. Problem Statement

Students at university may not have a clear understanding of the medical-submission process as to how to obtain a reference number, what documents are needed for submission, where completed documents should be turned in and what information should be verified prior to submission. Information gaps can lead to re-queries, missing information and unnecessary delays. A simple intelligent support system therefore is required that can classify common questions, give consistent administrative advice and determine possible missing documents without interfering with all the official decisions by the authorized staff.

4. Objectives

4.1 General Objective

To create an AI-based student support system to categorize medical-submission questions, give appropriate next-step instructions and enable the students to see if any requirements were missing before the official review.

4.2 Specific Objectives

1. Create a clear web interface for medical-submission guidance.
2. Build a labelled dataset of realistic but fictional student questions.
3. Convert text into numerical features using TF-IDF.
4. Train a Logistic Regression model to classify five request categories.
5. Display a probability-based confidence score and mark uncertain predictions for referral.
6. Provide category-specific explanations and recommended actions.
7. Offer the official request form and a cover-letter template for preparation.
8. Provide a structured seven-item submission checklist.
9. Use OCR and rule-based reasoning to flag possible omissions in uploaded document images.
10. Test the classifier, user flow and document-checking logic using sample inputs.

5. Selected AI Techniques

5.1 Natural Language Processing

Natural Language Processing is used to handle the question entered by the student. NLP is not used to create a new medical or administrative decision in MediSub. It helps prepare the normal text for the trained classifier to recognize the type of guidance needed. This will ensure that the response is managed and tied to the approved category guidance in the application.

5.2 TF-IDF Vectorization

The term TF-IDF, or Term Frequency–Inverse Document Frequency, converts the student question into numerical features that the classifier can process. MediSub uses single words and two-word phrases and will convert text to lower case. Therefore, the following expressions can help to distinguish the 5 categories: “reference number”, “required documents” and “where should I submit”.

5.3 Logistic Regression Classification

MediSub uses the supervised classification model Logistic Regression. Gets the TF-IDF features and outputs one of five categories: PROCESS_GUIDANCE, REFERENCE_NUMBER_HELP, DOCUMENT_REQUIREMENTS, MA_SUBMISSION_HELP or OUT_OF_SCOPE. A short answer and recommended next action is then linked to the predicted category in the Streamlit application.

5.4 Confidence-Based Referral

The largest class probability is used as the confidence score in the application. In MediSub, a threshold of 0.60 is used. If the value is lower, the page remains the same and will also display the suggested category with a warning message for the student requiring them to fill it out in detail or ask for permission from authorised staff.

5.5 Optical Character Recognition

Tesseract OCR reads visible text from JPG, JPEG or PNG document images. MediSub corrects image orientation, converts the page to grayscale, applies automatic contrast, increases contrast and sharpens the image before recognition. The extracted text is used only for completeness checks. OCR cannot confirm that a signature, stamp or medical statement is genuine.

5.6 Rule-Based Reasoning

Document completeness is checked with explicit rules. MediSub searches the OCR text for required headings, sections and a registration-number pattern. It combines those results with the user’s manual confirmations for signatures, stamps and approvals. The checker returns Ready for staff review, Needs correction or Unreadable. These labels support preparation and never represent official approval.

6. Justification of AI Techniques

Technique	Reason for selection
TF-IDF	Works well with a small text dataset, runs quickly and makes important words and phrases measurable.
Logistic Regression	Supports multi-class classification, provides probabilities, is fast to train and is easier to explain than a deep neural network.
Confidence threshold	Prevents low-confidence predictions from being presented without a clear warning.
Tesseract OCR	Provides an open-source method for extracting visible text from scanned or photographed documents.
Rule-based reasoning	Matches fixed administrative requirements that can be expressed as clear conditions and warnings.
Human review	Keeps official responsibility with authorized staff and controls the risk of automated errors.

Table 1: Justification of AI Techniques

Deep learning was not necessary for the current dataset. After cleaning, only 153 questions remained and the categories depend strongly on specific administrative vocabulary. A compact TF-IDF and Logistic Regression pipeline provides a suitable baseline without requiring high computing power or a large training corpus. Fuzzy logic, genetic algorithms and search algorithms were not used because the final problem is primarily text classification and rule-based completeness checking.

7. System Design

7.1 Overall Architecture

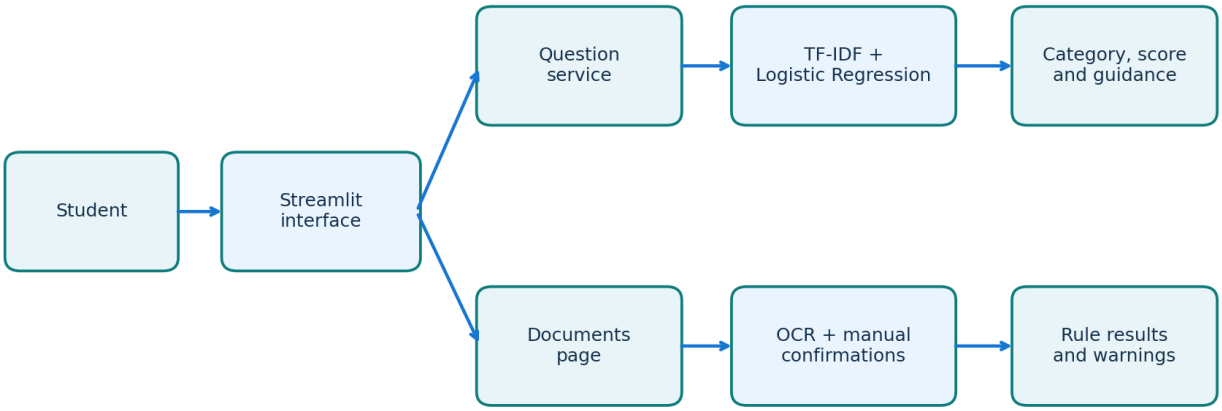


Figure 1: Overall MediSub architecture

The Streamlit interface is the central integration point. Question requests are passed to question_service.py, which loads the saved Joblib pipeline and returns a category, confidence value and referral flag. Document requests are handled by the Documents page. Uploaded images go through OCR and the appropriate document rules. The result is guidance rather than an administrative decision.

7.2 AI Classification Flow

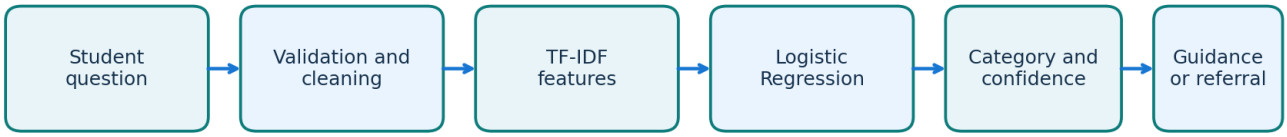


Figure 2:Question-classification workflow

7.3 Document Preparation and Checking Flow

The document flow begins when the student downloads the templates, completes the required sections and obtains the required signatures or approvals. Clear page images are then uploaded. Tesseract OCR extracts visible text and the document rules combine that text with the manual confirmations. The system finally displays passed checks, possible omissions and a readiness status for staff review.

7.4 Inputs and Outputs

Component	Inputs	Outputs
Question classifier	Question text up to 500 characters	Predicted category, confidence score, guidance, next action and uncertainty warning
Submission checklist	Seven checkbox answers	Completion count, progress bar and remaining requirement warning
Document checker	Request-form pages, cover letter, certificate/report and manual confirmations	Passed checks, warnings, manual-review notes and readiness status

Table 2:Inputs and Outputs

8. Implementation Details

8.1 Technology Stack

Technology	Use
Python	Main implementation language
Streamlit	Five-page interactive web interface
pandas	CSV loading and dataset cleaning
scikit-learn	TF-IDF, Logistic Regression, train/test split and evaluation
Joblib	Saving and loading the trained pipeline
Jupyter Notebook	Training, evaluation and experiment record
Pillow	Image opening and preprocessing
Tesseract and pytesseract	OCR text extraction
Matplotlib and Seaborn	Confusion-matrix visualization
Git and GitHub	Version control and team collaboration

Table 3:Inputs and Outputs

8.2 Dataset Preparation

The original CSV contained 201 rows with the columns id, question and category. The dataset had no missing values, but 48 questions were duplicates. Cleaning kept the question and category columns, removed blanks, standardized category names, removed duplicate question text and reset the row index. The final dataset contained 153 questions and remained almost balanced across all five categories.

Category	Before cleaning	After cleaning
PROCESS_GUIDANCE	40	31
REFERENCE_NUMBER_HELP	41	30
DOCUMENT_REQUIREMENTS	40	31

MA_SUBMISSION_HELP	40	30
OUT_OF_SCOPE	40	31
Total	201	153

Table 4: Dataset Preparation

8.3 Training and Saving the Model

The cleaned data were divided using an 80:20 stratified split with random_state=42. This produced 122 training questions and 31 testing questions. Stratification kept each class represented in the test set. The model pipeline was then fitted on the training data and saved to `models/medical_classifier.joblib.`

```
model = Pipeline([
    ('tfidf', TfidfVectorizer(lowercase=True, ngram_range=(1, 2),
min_df=1)),
    ('classifier', LogisticRegression(max_iter=1000,
random_state=42))
])
model.fit(X_train, y_train)
joblib.dump(model, '../models/medical_classifier.joblib')
```

8.4 Prediction Service

question_service.py validates the input, rejects an empty question and limits text to 500 characters. It loads the trained pipeline, predicts the class and obtains class probabilities. The returned dictionary contains category, confidence and needs_referral. Separating this logic from app.py keeps the interface code simpler.

```
category = model.predict([clean_question])[0]
probabilities = model.predict_proba([clean_question])[0]
confidence = float(probabilities.max())
needs_referral = confidence < 0.60
```

8.5 Streamlit Interface

The application uses horizontal radio navigation with five pages. Home introduces the service and provides quick actions. Ask a Question displays the model result, confidence bar, explanation and recommended next action. Documents contains preparation and upload tabs. Checklist tracks seven requirements. About and Help explains the system scope, privacy notice and limitations. Custom CSS in styles.py provides the white, teal and violet visual design and responsive layout.

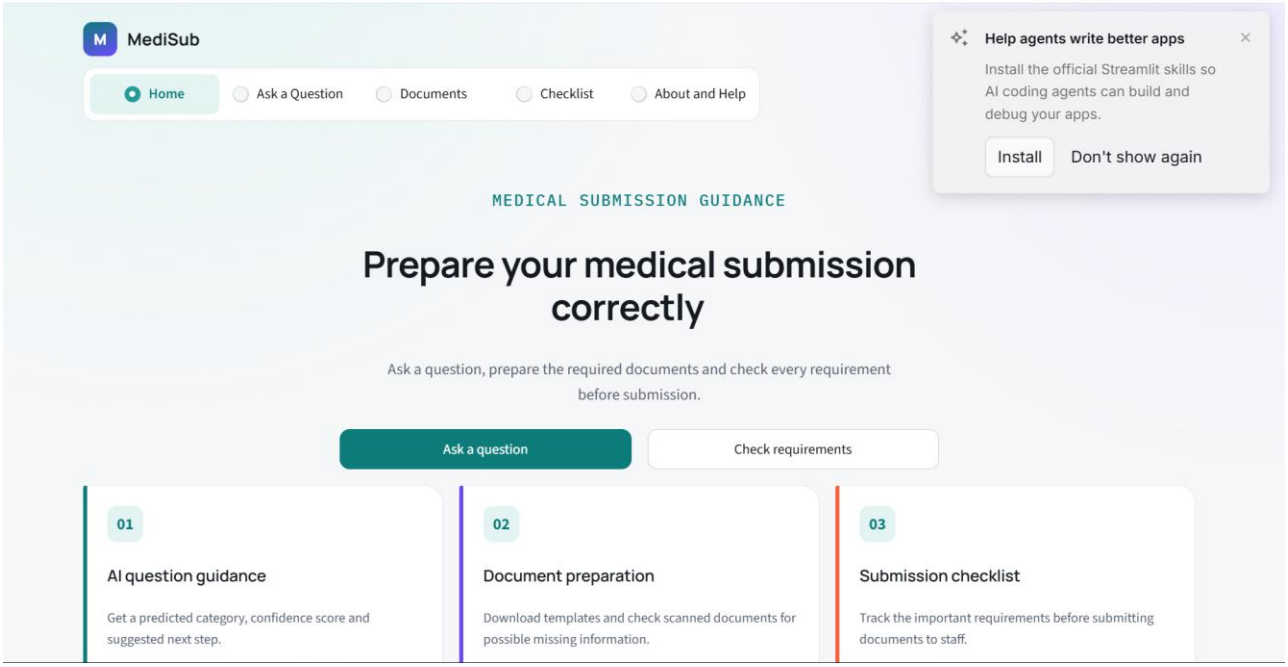


Figure 3:MediSub home page

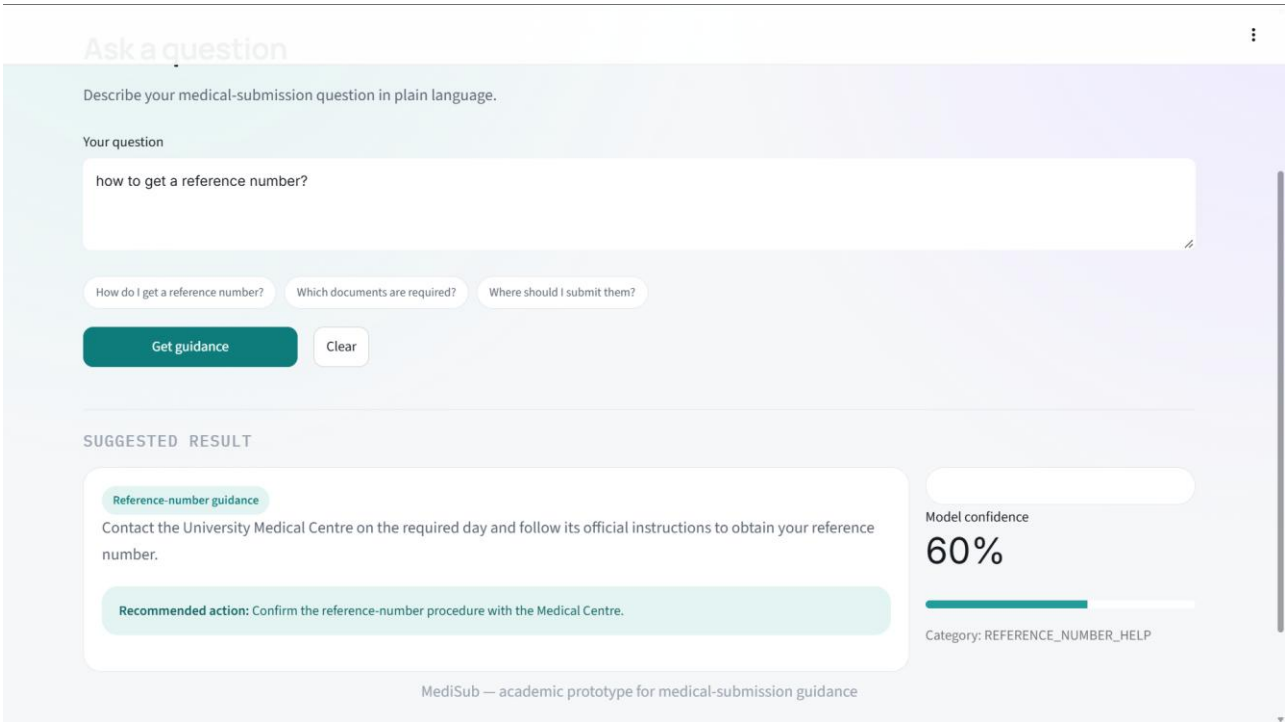


Figure 4:Question category and confidence result

8.6 Submission Checklist

The checklist records seven preparation confirmations. It covers the reference number, student sections of the request form, cover-letter signature, official signatures or stamps, supporting medical evidence, readability of names and dates and confirmation of the correct submission office. The completion count and progress bar update immediately.

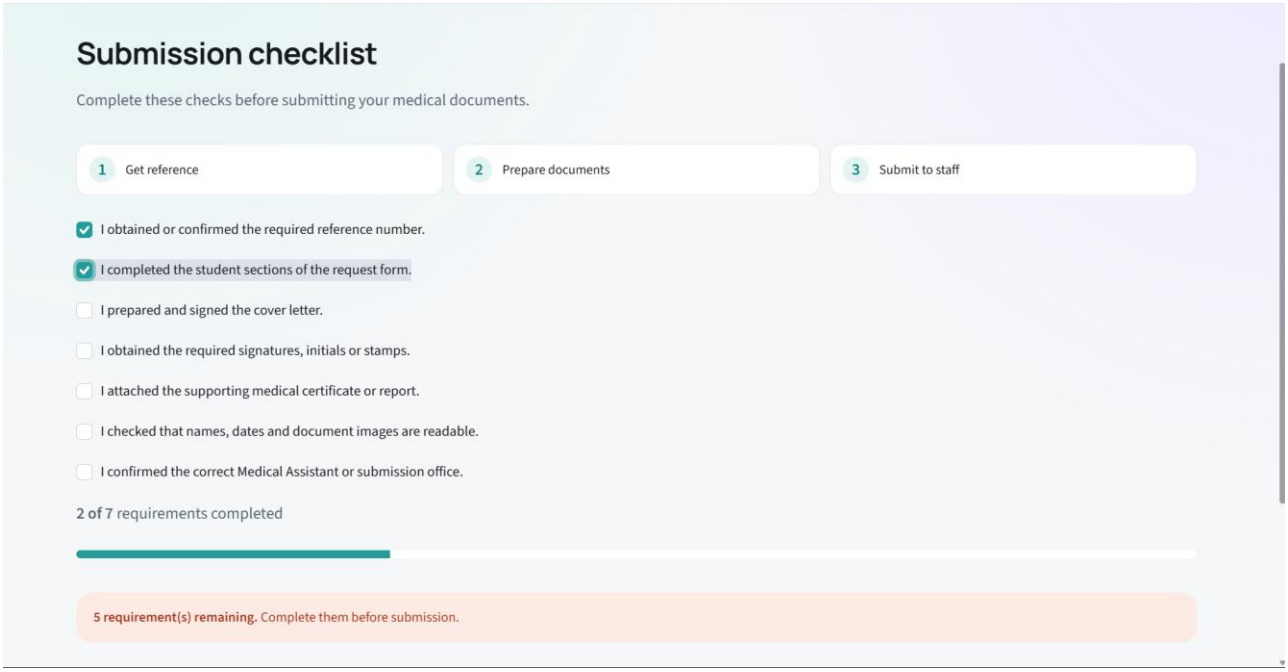


Figure 5:Submission checklist page

8.7 Document Preparation and OCR Checker

If the files are available on the downloads page, the Medical Approval Request Form as well as the cover-letter template will be available on the Documents page. The upload tab is able to upload a separate picture of the request form, one picture of the routed cover letter and one picture of the medical certificate/report. Manual confirmation is also provided on each of the documents as they cannot achieve the level of certainty they need based on visual evidence.

The final checker preprocess the image and asks Tesseract to recognize using page segmentation mode 6. It then uses document-specific rules. The request form includes text readability, form wording and 5 manual requirements. The cover letter is tested for readable text, medical-submission wording and four confirmations. Three confirmations, medical-document wording and readable text are checked in the certificate/report. All results contain staff-only checks, meaning a positive automated result won't be considered to be an approval.

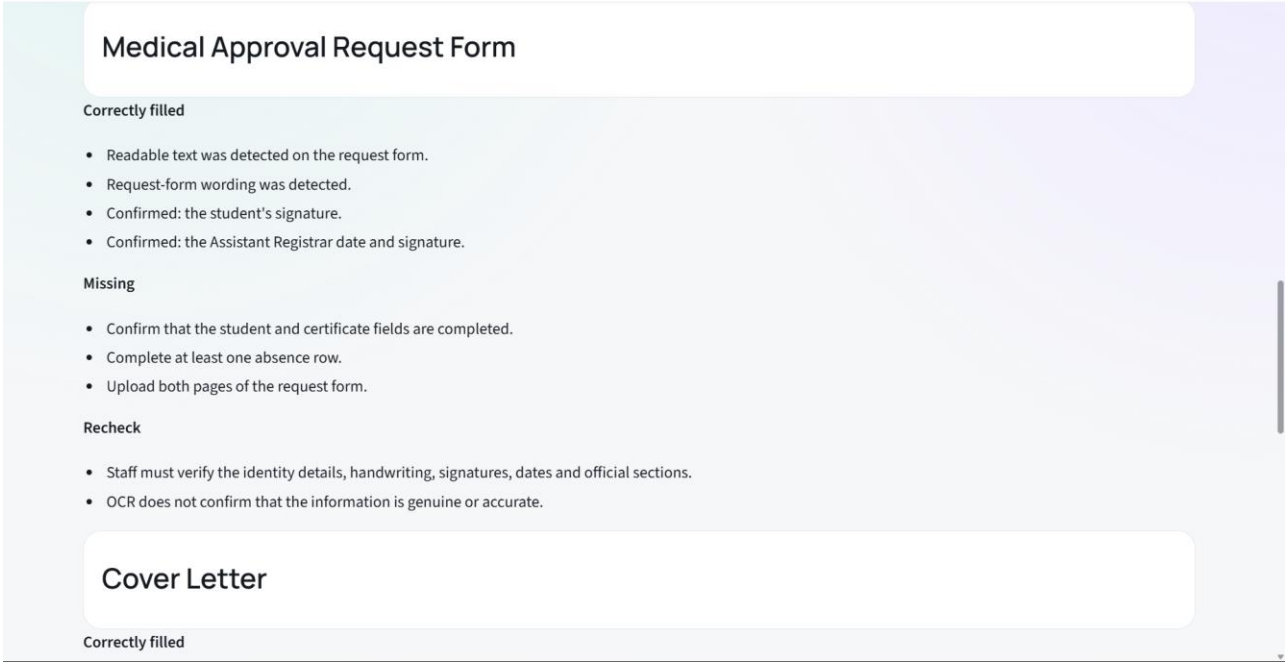


Figure 6:Document upload and OCR checking result

8.8 Privacy and Safety Messages

During prototype demonstrations, users are required to use fictional or redacted documents on the interface. Also it says that the system is unable to authenticate signatures, validate evidence or approve a submission. The messages appear at the place where users ask questions or upload files.

9. Testing and Results

9.1 Dataset Validation

```
    cleaned_df = (
        df.drop_duplicates(subset=["question"])
        .reset_index(drop=True)
    )

    print("Rows after cleaning:", len(cleaned_df))

    print("\nQuestions in each category:")
    print(cleaned_df["category"].value_counts())
```

[5] ✓ 0.1s

```
Original dataset rows: 241
Duplicate questions found: 48
Rows after cleaning: 193

Questions in each category:
category
PROCESS_GUIDANCE          31
DOCUMENT_REQUIREMENTS     31
OUT_OF_SCOPE              31
REFERENCE_NUMBER_HELP     30
MA_SUBMISSION_HELP        30
 PROCESS_GUIDANCE         10
 REFERENCE_NUMBER_HELP    10
 DOCUMENT_REQUIREMENTS    10
 OUT_OF_SCOPE             10
Name: count, dtype: int64
```

Figure 7:Data Set Validation

9.2 Model Evaluation

	precision	recall	f1-score	support
DOCUMENT_REQUIREMENTS	0.89	1.00	0.94	8
MA_SUBMISSION_HELP	1.00	1.00	1.00	6
OUT_OF_SCOPE	0.83	0.62	0.71	8
PROCESS_GUIDANCE	0.70	0.78	0.74	9
REFERENCE_NUMBER_HELP	1.00	1.00	1.00	8
accuracy			0.87	39
macro avg	0.88	0.88	0.88	39
weighted avg	0.87	0.87	0.87	39

Figure 8:Classification report for the five categories1

```python
from sklearn.metrics import classification_report

y_pred = model.predict(X_test)

print("Classification Report:\n")
print(
    classification_report(
        y_test,
        y_pred,
        digits=2
    )
)
```

[23] ✓ 0.0s

```
Classification Report:

                      precision    recall  f1-score   support

DOCUMENT_REQUIREMENTS       0.89      1.00      0.94         8
   MA_SUBMISSION_HELP       1.00      1.00      1.00         6
        OUT_OF_SCOPE       0.83      0.62      0.71         8
     PROCESS_GUIDANCE       0.70      0.78      0.74         9
REFERENCE_NUMBER_HELP       1.00      1.00      1.00         8

             accuracy                           0.87        39
            macro avg       0.88      0.88      0.88        39
         weighted avg       0.87      0.87      0.87        39
```

Figure 9:Classification report for the five categories2

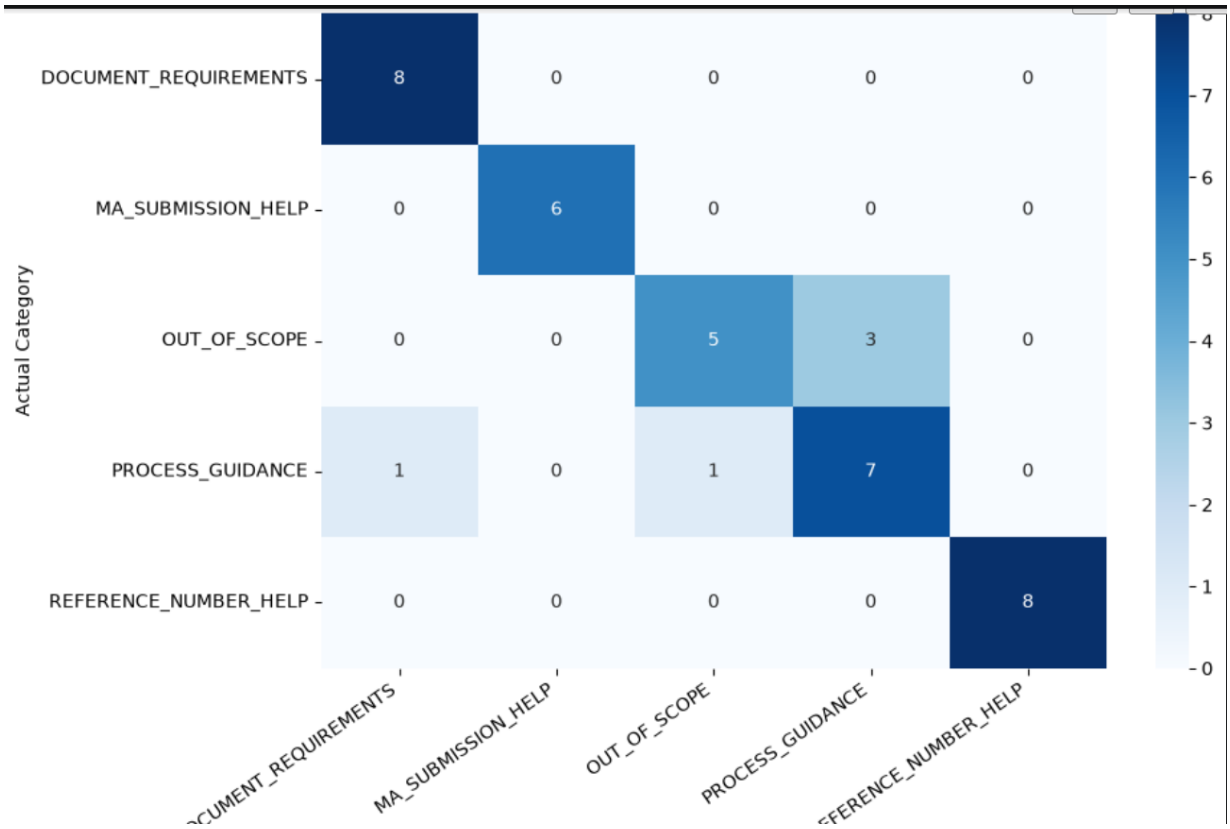


Figure 10:Confusion matrix for the 39-question test set

.

The test classification was perfect in the confusion matrix in terms of the MA_SUBMISSION_HELP and REFERENCE_NUMBER_HELP. Of the seven document requirement questions, one was not actually recognized as such, but two were out of scope and predicted as document requirements. 75% of the 6 OUT_OF_SCOPE questions was correct. Two were forecast as process guidance and one forecast as document requirements. That is the primary bug that was found in the test set.

9.3 Manual Question Tests

Question	Predicted category	Confidence	Application action
What documents should I submit?	DOCUMENT_REQUIREMENTS	43.33%	Show guidance and low-confidence warning
How do I obtain my medical reference number?	REFERENCE_NUMBER_HELP	71.08%	Show reference-number guidance
Where should I submit my completed medical forms?	MA_SUBMISSION_HELP	30.54%	Show submission guidance and low-confidence warning
What is the medical submission process?	PROCESS_GUIDANCE	59.86%	Show process guidance and low-confidence warning
How can I change my Wi-Fi password?	PROCESS_GUIDANCE	28.58%	Show low-confidence warning and advise the user to confirm with staff

Table 5:Manual Question Tests

The following examples illustrate the need for a confidence-based referral. The first four questions went to the anticipated categories to be submitted to the medical world. Three of them, however, had a confidence score below 60% and would need to be confirmed through authorized staff. The confidence level for the referent numbers question was 71.08% and it could be shown without warning of low confidence. This unrelated, Wi-Fi question was miscategorized with low confidence of 28.58% as PROCESS_GUIDANCE. The system thus presents an uncertainty warning rather than confidence as guidance. This behaviour could be improved with the addition of a wider variety of unrelated questions into the OUT_OF_SCOPE category.

9.4 Document Checker Tests

Test	Expected result	Observed result	Status
Complete request form, letter and certificate confirmations	Ready for staff review	All three objects returned Ready for staff review	Pass
Request form with student signature unchecked	Needs correction and signature warning	Needs correction with signature warning	Pass
Unreadable certificate text	Needs correction	Needs correction with readability and identification warnings	Pass

Table 6:Document Checker Tests

All three implemented tests passed for the rule layer. The runtime behaviour of OCR still relies on the correct installation of Tesseract and pytesseract locally. A production evaluation should have a controlled set of clear, blurred, handwritten and low-light document images; it should report the character or field recognition accuracy. This academic test should not be done by using real medical records unless approved and protected.

9.5 Functional Test Summary

Function	Expected behaviour	Result
Empty question	Warning shown	Implemented
Question over 500 characters	Validation error	Implemented
Valid question	Category, confidence and guidance shown	Implemented
Low-confidence prediction	Referral warning shown	Implemented
Navigation	One-click page change	Implemented
Checklist	Count and progress update	Implemented
Missing upload	Document-specific error shown	Implemented
Document rules	Passed, warnings and manual checks shown	Implemented and unit-tested

Table 7:Functional Test Summary

10. Limitations and Discussion

The selected pipeline has an 87.1% test accuracy, which is a decent result as the pipeline distinguishes between common medical-submission queries reasonably well. The test set is, however, limited to 31 questions, so the result is a prototype result and not representative of production. Questions are formulated by the group and can be repeated sentence structures. Duplicate data have been eliminated prior to training, but the language used in the data may be less representative of actual student language.

The lowest score is Out of scope (0.50). The Wi-Fi example illustrates the problem: although the subject is not related to the question, the wording is familiar. The confidence threshold helps to reduce risk, but does not change the predicted label to OUT_OF_SCOPE. A future version should incorporate a broader set of unrelated questions, study the threshold calibration for studying and incorporate a separate in-scope detector prior to the five-classifier. The model offers more of a guideline on the process of finding a category answer than a new answer itself. This type of design can be used to enhance consistency and simplify auditing, but it is not suitable for all special cases. Guidance is embedded in code and needs to be changed if the official process changes.

OCR is light-sensitive, blurry, page angle-sensitive, handwriting and cropping-sensitive. For appropriate signatures, stamps and approvals, the manual confirmation is used by the checker, since basic OCR cannot safely confirm these features. A "Ready for staff review" result does not indicate that any of the configured warnings were not triggered. This is not a "good" or "approved" answer and does not show the validity of the evidence. SQLite and a staff dashboard were in the initial list

of proposed items but are not on the final 5 pages of the student application (verified). With them excluded, the prototype was kept to a minimum of 10 weeks to demonstrate the required AI. Case history, feedback review and authorized staff management are still potential future enhancements.

11. Ethical and Social Considerations

11.1 Privacy and Data Minimization

Medical documents contain names, registration numbers, diagnoses, treatment information and signatures. During demonstrations, however, the prototype asks users to use fictional or redacted documents as a guide. The training data includes questions that are not real or medical records. A deployed version should gather only beneficial information, encrypt communications and have a short retention time.

11.2 Accuracy and Reliability

Even if a model shows a confidence value it can be wrong. The confidence level isn't evidence that the answer is right. If the score is low, the application shows some uncertainty and will put the students in the official staff. OCR output may also be incomplete and warnings are issued by the checker, not decisions.

11.3 Bias

The vocabulary, grammar and question types in a dataset created by a small group might be favored. Predictions may be less high quality for students who use unfamiliar words and/or another language. Counting the number of items in each category does not eradicate language bias.

11.4 Human Responsibility

MediSub supports administration; it does not replace university staff. Authorized officers remain responsible for confirming the procedure, examining original evidence and making the final decision. The system also avoids clinical diagnosis and treatment advice.

11.5 Misuse and Security

Future data collection should involve other types of writing and be evaluated for privacy and fairness. It is important to remember that MediSub is for administration and not to replace staff members of the university. Authorized officers still have the responsibility to establish the procedure and to review the original evidence and decide. The system also does not include clinical diagnosis and clinical treatment suggestions.

12. Conclusion

MediSub is an academic prototype for medical-submission guidance and is a comprehensive system. It integrates a trained text classifier with a five-page web interface, category-based answers, next step recommendations, confidence warnings, document preparation support and a structured checklist and completeness checker using OCR. The project thus exemplifies classification, NLP, probability-based decision support, OCR and rule-based reasoning in the same unified flow of students.

After cleaning the original data set of 201 questions, the final TF-IDF and Logistic Regression model was trained on 122 questions and tested on 31 questions. It had an accuracy of 87.1% and a weighted F1-score of 0.86. The findings are promising for a small trial, especially for questions that require a reference number and/or submission location. Weaker OUT_OF_SCOPE recall and low confidence on some manual examples, however, demonstrate the need for staff confirmation to be maintained concurrently.

The finished solution has a primary goal: to guide a student in understanding what to do before submitting documents. MediSub does not provide reference numbers, validate medical claims or approve a claim. Maintaining those boundaries visible will help keep the system safe and realistic for use in universities.

13.References

[1] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On calibration of modern neural networks,” in Proc. 34th Int. Conf. Mach. Learn. (ICML), vol. 70, pp. 1321–1330, 2017. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html. [Accessed: Oct. 10, 2026].

[2] Python Software Foundation, “pickle — Python object serialization,” Python 3.7.3 Documentation, 2019. [Online]. Available: https://docs.python.org/3/library/pickle.html. [Accessed: Oct. 10, 2026].

[3] Streamlit, “Streamlit documentation,” 2024. [Online]. Available: https://docs.streamlit.io/. [Accessed: Oct. 10, 2026].

[4] Tesseract OCR, “tesseract-ocr/tesseract,” GitHub, Oct. 20, 2019. [Online]. Available: https://github.com/tesseract-ocr/tesseract. [Accessed: Oct. 10, 2026].

[5] B. Villon, “School clinic medical record system,” Academia.edu. [Online]. Available: https://www.academia.edu/9854472. [Accessed: Oct. 17, 2026].

[6] Pipilika Soft, “Medical college student management system (ERP),” Pipilika Soft, Sept. 29, 2019. [Online]. Available: https://pipilikasoft.com/medical-college-management-system/. [Accessed: Oct. 17, 2026].

14. Individual Contribution Sheets

Member 01:

Student Name	K.A.S. Nenuwari	**Registration No.**	D/BCS/25/0025
Degree / Intake	BCS / Intake 42	**Group No.**	51
Project Title	MediSub: An AI-Based Student Support System for Medical Submission Guidance		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Dataset preparation and expansion	Prepared and expanded the labelled medical-submission dataset with question variations, including attendance-shortage and illness-related scenarios.	Dataset preparation, domain review and CSV handling	Question records in data/medical_questions.csv
Dataset quality and balancing	Checked missing values, duplicate questions, category labels and category counts, then supported cleaning and maintaining a reasonably balanced dataset across the five classes.	Pandas validation, duplicate detection and class-count analysis	Quality-checking and category-count cells in notebooks/train_model.ipynb; data/medical_questions.csv
OCR document checking	Implemented OCR-based text extraction for uploaded JPG, JPEG and PNG images of medical-submission documents using image preprocessing and Tesseract.	Python, pytesseract, Tesseract OCR and text processing	extract_text() and _tesseract_command() in document_checker.py
Rule-based completeness checking	Implemented rule-based completeness checks for the request form, cover letter and medical certificate using normalized OCR text and required-item mappings.	Rule-based checking, text normalization and keyword matching	check_request_form(), check_cover_letter() and check_medical_certificate() in document_checker.py
Document guidance and limitations	Prepared document-checking guidance, status explanations and warnings so users can identify missing items without treating OCR results as document authentication or final approval.	Responsible-AI guidance, OCR limitations and staff-review rules	docs/DOCUMENT_RULES.md; document_checker.py; documents.py

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** __17/10/2026_____

Member 02:

Student Name	R.S.K.S. Rajasundara	**Registration No.**	D/BIT/24/0016
Degree / Intake	BIT / Intake 41	**Group No.**	51
Project Title	MediSub: An AI-Based Student Support System for Medical Submission Guidance		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
TF-IDF feature extraction	Configured TF-IDF to convert medical-submission questions into numerical features. Used lowercase processing with unigrams and bigrams to learn single words and useful two-word phrases.	NLP, TfidfVectorizer and scikit-learn Pipeline	TF-IDF pipeline in notebooks/train_model.ipynb
Logistic Regression classifier	Developed and trained the multi-class Logistic Regression model to classify questions into the five supported medical-submission categories.	Supervised learning, LogisticRegression and scikit-learn	Model-training cells in notebooks/train_model.ipynb
Model evaluation	Evaluated the trained model using unseen test questions and reviewed accuracy, precision, recall, F1-score and the confusion matrix for the five categories.	train_test_split, classification_report and confusion_matrix	Evaluation and confusion-matrix cells in notebooks/train_model.ipynb
Confidence handling	Used class probabilities to calculate the prediction confidence and applied a 0.60 threshold so uncertain results request confirmation from authorized staff.	predict_proba and confidence-threshold logic	CONFIDENCE_THRESHOLD and predict_question() in question_service.py
Reusable trained model	Saved the complete TF-IDF and Logistic Regression pipeline with Joblib so the application could load the trained model without retraining it during every use.	Joblib and model persistence	Model-saving cell in notebooks/train_model.ipynb; models/medical_classifier.joblib

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 17.10.2026

Member 03:

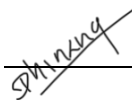
Student Name	M.K.F.G. Dhinany	**Registration No.**	D/BIT/24/0030
Degree / Intake	BIT / Intake 41	**Group No.**	51
Project Title	MediSub: An AI-Based Student Support System for Medical Submission Guidance		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Manual test-case preparation	Prepared a manual test-case structure covering navigation, questions, confidence results, uploads, OCR, downloads, checklist progress, warnings and responsive layout.	Functional test planning and expected-result design	data/rule_test_cases.csv; reports/MediSub_Manual_Functional_Test_Cases.docx
Setup documentation	Prepared the environment, dependency, model-training and application-startup instructions needed for another user to install and run the system.	Technical writing and setup verification	README.md; requirements.txt; notebooks/train_model.ipynb
Technical workflow documentation	Documented the document-preparation and checking workflow, statuses, expected document contents, OCR limits and staff-review requirements.	Technical documentation and process description	docs/DOCUMENT_RULES.md; README.md
System limitations and privacy	Documented that MediSub cannot provide medical advice, authenticate evidence, issue reference numbers or approve submissions and that authorized staff must make final decisions.	Responsible-AI, privacy and limitation documentation	Limitations in app.py, document_checker.py and docs/DOCUMENT_RULES.md
GitHub repository organization	Organized application code, datasets, notebooks, model files, documentation, downloads and reports into clear repository areas.	GitHub, repository structure and file organization	README.md; data/; notebooks/; models/; docs/; downloads/; reports/

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 17.10.2026

Member 04:

Student Name	A.S. Fernando	**Registration No.**	D/DBA/25/0017
Degree / Intake	DBA / Intake 42	**Group No.**	51
Project Title	MediSub: An AI-Based Student Support System for Medical Submission Guidance		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Streamlit interface and navigation	Developed navigation between Home, Ask a Question, Documents, Checklist and About and Help, including the main user-facing controls and page transitions.	Python, Streamlit and session state	Page functions, navigation and change_page() in app.py
Buttons, checklist and Help content	Implemented page buttons, checklist selections, completion counting, progress feedback and the About and Help content used to guide students through the system.	Streamlit callbacks, checkboxes, progress and page content	Home controls, checklist_page() and about_page() in app.py
Document interface and downloads	Implemented the user-facing document page with image upload controls, manual confirmation fields, document results, warnings and downloadable request-form and cover-letter templates.	Streamlit file uploaders, download buttons and result presentation	render_documents_page() and download controls in documents.py
User-facing validation	Integrated clear messages for empty questions, questions longer than 500 characters, missing document uploads and supported image-extension restrictions.	Input validation, exception handling and extension filtering	ask_page() in app.py; length rule in question_service.py; upload controls in documents.py
Responsive styling and feedback	Implemented responsive layout, page alignment, consistent visual styling and user-facing error, warning and success messages across the application.	Responsive design, HTML, CSS and Streamlit messages	apply_styles() and media rules in styles.py; .streamlit/config.toml; messages in app.py and documents.py

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** ___17/10/2026_____